

→JS-8

→Array methods

Array Methods

- `forEach`
- `map`
- `filter`
- `some`
- `every`
- `reduce`

forEach

`arr.forEach`(some function definition or name);

forEach

```
let arr = [1, 2, 3, 4, 5];

function print(el) {
  console.log(el);
}

arr.forEach(print);

// OR

arr.forEach(function(el) {
  console.log(el);
});
```

```
app.js > ...
1  let arr = [1, 2, 3, 4, 5];
2
3  let print = function (el) {
4    console.log(el);
5  };
6
7  arr.forEach(print);
8
```

1
2
3
4
5
> |

```
let arr = [1, 2, 3, 4, 5];
arr.forEach(function(el) {
  console.log(el);
});
```

```
p.js > ...
let arr = [
  {
    name: "aman",
    marks: 95,
  },
  {
    name: "shradha",
    marks: 94.4,
  },
  {
    name: "rajat",
    marks: 92,
  },
];
arr.forEach((student) => {
  console.log(student);
});
```

```
▶ {name: 'aman', marks: 95}
▶ {name: 'shradha', marks: 94.4}
▶ {name: 'rajat', marks: 92}
```

```
p.js > arr.forEach() Callback
let arr = [
  {
    name: "aman",
    marks: 95,
  },
  {
    name: "shradha",
    marks: 94.4,
  },
  {
    name: "rajat",
    marks: 92,
  },
];
arr.forEach((student) => {
  console.log(student.marks);
});
```

```
95
94.4
92
```

→ Map

Map

```
let newArr = arr.map(some function definition or name);
```

```
let num = [1, 2, 3, 4];

let double = num.map(function(el) {
  return el*2;
});
```

```
> double
< ▶ (4) [2, 4, 6, 8]
```

```
let num = [1, 2, 3, 4];

let double = num.map((el) => {
  return el * 2;
});
```

```
let double = num.map((el) => {});
```

No issues

```
> double
< ▶ (4) [undefined, undefined, undefined, undefined]
```

p.js > gpa > students.map() callback

```
let students = [
  {
    name: "aman",
    marks: 95,
  },
  {
    name: "shradha",
    marks: 94.4,
  },
  {
    name: "rajat",
    marks: 92,
  },
];

let gpa = students.map((el) => {
  return el.marks / 10;
});
```

```
> gpa
< ▼ (3) [9.5, 9.440000000000001, 9.2] ⓘ
  0: 9.5
  1: 9.440000000000001
  2: 9.2
  length: 3
  ▶ [[Prototype]]: Array(0)
```

Filter

`let newArr = arr.filter(some function definition or name);`



```
let nums = [2, 4, 1, 5, 6, 2, 7, 8, 9];

let even = nums.filter( (num) => (num % 2 == 0) );
```

```
let nums = [1, 2, 3, 4, 7, 8, 2, 9, 10, 12, 11];
let ans = nums.filter((el) => {
  return el % 2 == 0; //even -> true, odd -> false
});
```

```
ans
> (6) [2, 4, 8, 2, 10, 12]
```

```
// });

let nums = [1, 2, 3, 4, 7, 8, 2, 9, 10, 12, 11];
let ans = nums.filter((el) => {
  return el % 2 != 0; //even -> true, odd -> false
});
```

```
> ans
< ▶ (5) [1, 3, 7, 9, 11]
```

Every

true
false

AND

Returns true if **every** element of array gives true for some function. Else returns false.

`arr.every(some function definition or name);`

↓ true ↓ false

```
[1, 2, 3, 4].every( (el) => (el%2 == 0));  
false  
[2, 4].every( (el) => (el%2 == 0));  
true
```

```
> [2, 4, 6].every((el) => el%2 == 0);  
< true  
> [2, 4, 6, 8].every((el) => el%2 == 0);  
< true  
> [2, 4, 6, 8, 1].every((el) => el%2 == 0);  
< false  
> |
```

```
false  
[2, 4, 6, 8, 1].every((el) => el%2 != 0);  
false  
[3, 5, 1].every((el) => el%2 != 0);  
true
```

→ Reduce

Reduce

Reduces the array to a single value

`arr.reduce( reducer function with 2 variables for (accumulator, element) );`

```
[1, 2, 3, 4].reduce( (res, el) => (res+el) );  
10
```

Reduce

Reduces the array to a single value

some - T/F
every - T/F

single

[1, 2, 3, 4, 5, 6]

`arr.reduce( reducer function with 2 variables for (accumulator, element) );`

[1, 2, 3]

① acc, el → return

② acc, el → return

③ acc, el → return

Reduced

```
[1, 2, 3, 4].reduce( (res, el) => (res+el) );  
10
```

el, a, b

```
8  
9 let nums = [1, 2, 3, 4];  
10 let finalVal = nums.reduce((res, el) => res + el);  
11 console.log(finalVal);  
12
```

[1, 2, 3, 4]
↑ ↑ ↑ ↑

`let finalVal = nums.reduce( (res, el) => res + el );`
↑↑↑

(0, 1) ⇒ 1

(1, 2) ⇒ 3

(3, 3) ⇒ 6

(6, 4) ⇒ 10

finalVal

```
let nums = [1, 2, 3, 4];
let finalVal = nums.reduce((res, el) => {
  console.log(res);
  return res + el;
});
console.log(finalVal);
```

1

3

6

10

Reduce

Finding **Maximum** in an array

```
let nums = [2, 3, 4, 5, 3, 4, 7, 8, 1, 2];

let result = nums.reduce( (max, el) => {
  if(el > max) {
    return el;
  } else {
    return max;
  }
});
```

```
let arr = [1, 4, 2, 5, 6, 7, 2, 9, 2];

// let max = -1;

// for (let i = 0; i < arr.length; i++) {
//   if (max < arr[i]) {
//     max = arr[i];
//   }
// }

// console.log(max);

let max = arr.reduce((max, el) => {
  if (max < el) {
    return el;
  } else {
    return max;
  }
});
```

9

→ Default parameters

Default Parameters

Giving a default value to the arguments

```
function func (a, b = 2) {  
  //do something  
}
```

```
function sum(a, b = 3) {  
  return a + b;  
}
```

```
sum(2); //5
```

```
pp.js > sum  
function sum(a, b = 2) {  
  return a + b;  
}
```

```
> sum(1);  
3  
>  
> sum(1, 4);  
5  
>
```

```
pp.js > sum  
function sum(a = 2, b) {  
  return a + b;  
}
```



```

app.js > ...
1  function sum(a = 2, b) {
2    return a + b;
3  }
4
5  sum(1, 3); //4
6  sum(1); //a = 1, b = undefined
7

```

```

top Filter
sum(1, 3); //4
sum(1); //a = 1, b = undefined
NaN
sum(1, 3); //4
4
sum(1); //a = 1, b = undefined
NaN

```

→ Spread

Spread

Expands an iterable into multiple values

```

function func (...arr) {
  //do something
}

```

```

> console.log(..."apnacollege");
a p n a c o l l e g e

```

```

> let arr = [1, 2, 3, 4, 5];
< undefined
> Math.min(...arr);
< 1
> console.log(...arr);
1 2 3 4 5

```



```
> Math
< ▶ Math {abs: f, acos: f, acosh: f, asin: f, asinh: f, _}

> Math.min
< f min() { [native code] }

> Math.min(1, 2, 3);
< 1

> Math.min(1, 2, 3, 1, 2, 3, 0);
< 0

> Math.min(1, 2, 3, 1, 2, 3, 0, 1, 2, 3, 1, 2, 3, 0);
< 0

> let arr = [1, 2, 3, 1, 2, 3, 0, 1, 2, 3, 1, 2, 3, 0];
< undefined

> arr
< ▶ (14) [1, 2, 3, 1, 2, 3, 0, 1, 2, 3, 1, 2, 3, 0]

> Math.min(...arr);
< 0

> arr.push(-1);
< 15

> Math.min(...arr);
< -1

> Math.max(...arr);
< 3

> console.log(arr);
  ▶ (15) [1, 2, 3, 1, 2, 3, 0, 1, 2, 3, 1, 2, 3, 0, -1]
< undefined

> console.log(...arr);
  1 2 3 1 2 3 0 1 2 3 1 2 3 0 -1
< undefined

> console.log(1, 2, 3, 1);
  1 2 3 1
< undefined

> console.log("apnacollege");
  apnacollege
< undefined

> console.log(..."apnacollege");
  a p n a c o l l e g e
< undefined

>
```

→ Spread with Array literals

Spread

with Array Literals

```
> let arr = [1, 2, 3, 4, 5];  
< undefined  
> let newArr = [...arr];  
< undefined  
> newArr  
< ▶ (5) [1, 2, 3, 4, 5]
```

```
> let chars = [..."hello"];  
< undefined  
> chars  
< ▶ (5) ['h', 'e', 'l', 'l', 'o']
```

```
pp.js > ...  
let arr = [1, 2, 3, 4, 5];  
let newArr = [...arr];
```

```
top Filter  
> arr  
< ▶ (5) [1, 2, 3, 4, 5]  
> newArr  
< ▶ (5) [1, 2, 3, 4, 5]  
> newArr.push(6);  
< 6  
> newArr  
< ▶ (6) [1, 2, 3, 4, 5, 6]  
> newArr  
< ▶ (6) [1, 2, 3, 4, 5, 6]  
> arr  
< ▶ (5) [1, 2, 3, 4, 5]  
> |
```

```
let odd = [1, 3, 5, 7, 9];  
let even = [2, 4, 6, 8, 10];  
let nums = [...odd, ...even];
```

nums

```
▶ (10) [1, 3, 5, 7, 9, 2, 4, 6, 8, 10]
```

→ Spread with Object literals

Spread

with Object Literals

```
let data = {  
  email: "ironman@gmail.com",  
  password: "abcd",  
};  
  
let dataCopy = { ...data, id: 123 };
```

```
const data = {  
  email: "ironman@gmail.com",  
  password: "abcd",  
};  
  
const dataCopy = { ...data };
```

```
> dataCopy  
< ▶ {email: 'ironman@gmail.com', password: 'abcd'}  
>
```

```
JS app.js > [?] dataCopy
```

```
1  const data = {  
2    email: "ironman@gmail.com",  
3    password: "abcd",  
4  };  
5  
6  const dataCopy = { ...data, id: 123, };  
7
```

top Filter

dataCopy

```
▶ {email: 'ironman@gmail.com', password: 'abcd', id: 123}
```

```
JS app.js > [?] obj2
```

```
1  const data = {  
2    email: "ironman@gmail.com",  
3    password: "abcd",  
4  };  
5  
6  const dataCopy = { ...data, id: 123, country: "India" };  
7  
8  let arr = [1, 2, 3, 4, 5]; //val  
9  let obj1 = { ...arr }; //obj -> key:val  
10  
11 let obj2 = {..."hello"};
```

obj2

```
▼ {0: 'h', 1: 'e', 2: 'l', 3: 'l', 4: 'o'} ⓘ
```

```
0: "h"
```

```
1: "e"
```

```
2: "l"
```

```
3: "l"
```

```
4: "o"
```

```
▶ [[Prototype]]: Object
```

```

1 function sum(...args) {
2   //arguments
3   for (let i = 0; i < args.length; i++) {
4     console.log("you gave us: ", args[i]);
5   }
6 }
7

```

```

> sum(1);
you gave us: 1
< undefined
> sum(1, 2);
you gave us: 1
you gave us: 2
< undefined
> sum(1, 2, 3, 4);
you gave us: 1
you gave us: 2
you gave us: 3
you gave us: 4
< undefined
>

```

```

function min(a, b, c, d) {
  console.log(arguments);
}

```

```

> min(1, 2, 3, 4);
Arguments(4) [1, 2, 3, 4, callee: f, Symbol(Symbol.iterator): f]
  0: 1
  1: 2
  2: 3
  3: 4
  ▶ callee: f min()
  length: 4
  ▶ Symbol(Symbol.iterator): f values()
  ▶ [[Prototype]]: Object
< undefined

```

```

function min() {
  console.log(arguments.length);
}

```

```

> min(1, 2, 3, 4);
4
< undefined
>

```

```

app.js > ...
1  function sum(...args) {
2      //arguments
3      for (let i = 0; i < args.length; i++) {
4          console.log("you gave us: ", args[i]);
5      }
6  }
7
8  function min() {
9      console.log(arguments);
10     console.log(arguments.length);
11     arguments.push(1);
12 }
13

```

```

> min(1, 2, 3, 4);
  ▶ Arguments(4) [1, 2, 3, 4, callee: f, Symbol(Symbol.iterator): f]
  4
  ▶ Uncaught TypeError: arguments.push is not a function
    at min (app.js:11:13)
    at <anonymous>:1:1

```

→ in arguments work like the arrays but we cannot apply all array methods on it

```

app.js > sum
function sum() {
  return arguments.reduce((sum, el) => sum + el);
}

```

```

  top  Filter
  > sum(1, 2, 3, 4);
  ✖ ▶ Uncaught TypeError: arguments.reduce is not a function
    at sum (app.js:2:20)
    at <anonymous>:1:1
  >

```

```
p.js >  sum
function sum(...args) {
   return args.reduce((sum, el) => sum + el);
}
```

```
> sum(1, 2, 3, 4);
< 10
> |
```

```

function min(...args) {
  return args.reduce((min, el) => {
    if (min > el) {
      return el;
    } else {
      return min;
    }
  });
}
```

```
 top Filter
> min(1, 2, 3, 4);
< 1
> min(12, 445, 123, -20);
< -20
>
```



```
function min(msg, ...args) {
  console.log(msg);
  return args.reduce((min, el) => {
    if (min > el) {
      return el;
    } else {
      return min;
    }
  });
}
```

```
> min("hello", 12, 445, 123, -20);
hello
< -20
>
```

→ Destructuring

Destructuring

Storing values of array into multiple variables

```
let names = ["tony", "bruce", "steve", "peter"];
let [winner, runnerup] = names;
console.log(winner, runnerup); // "tony" "bruce"
```

```
o.js > names
let names = ["tony", "bruce", "peter", "steve"];
let winner = names[0];
// let runnerup = names[1];
// let secondRunnerup = names[2];

let [winner, runnerup, secondRunnerup] = names;
```

```
top
> winner
< 'tony'
> runnerup
< 'bruce'
>
```

```
top
> others
< 'peter'
> |
```

```
app.js > [?] others
let names = ["tony", "bruce", "peter", "steve", "abc", "xyz", "pyq"];
// let winner = names[0];
// let runnerup = names[1];
// let secondRunnerup = names[2];

let [winner, runnerup, others] = names;
```

```
app.js > [?] others
let names = ["tony", "bruce", "peter", "steve", "abc", "xyz", "pyq"];
// let winner = names[0];
// let runnerup = names[1];
// let secondRunnerup = names[2];

let [winner, runnerup, ...others] = names;
```

```
top Filter
> others
< ▶ (5) ['peter', 'steve', 'abc', 'xyz', 'pyq']
>
```

→ Destructuring for objects

Destructuring

Objects

```
const student = {
  name: "karan",
  class: 9,
  age: 14,
  subjects: ["hindi", "english", "math", "science", "social studies"],
  username: "karan123",
  password: 1234,
};

const { username: user, password: pass } = student;

console.log(user); // "karan123"
```

```

pp.js > username
const student = {
  name: "karan",
  age: 14,
  class: 9,
  subjects: ["hindi", "english", "math", "science"],
  username: "karan@123",
  password: "abcd",
};
let { username, password } = student;

```

```

> username
< 'karan@123'

```

jcreddy2096@gmail

```

const student = {
  name: "karan",
  age: 14,
  class: 9,
  subjects: ["hindi", "english", "math", "science"],
  username: "karan@123",
  password: "abcd",
};
let { user, password } = student;

```

```

> user
< undefined
> |

```

```

pp.js > user
const student = {
  name: "karan",
  age: 14,
  class: 9,
  subjects: ["hindi", "english", "math", "science"],
  username: "karan@123",
  password: "abcd",
};
let { username: user, password } = student;

```

```

> username
> user
< 'karan@123'
> password
< 'abcd'
>

```

```
pp.js > [?] user
const student = {
  name: "karan",
  age: 14,
  class: 9,
  subjects: ["hindi", "english", "math", "science"],
  username: "karan@123",
  password: "abcd",
};

let { username: user, password: secret } = student;
```

```
> username
✖ ▶ Uncaught ReferenceError: username is not defined
  at <anonymous>:1:1
> password
✖ ▶ Uncaught ReferenceError: password is not defined
  at <anonymous>:1:1
> user
< 'karan@123'
>
```

```
> city
< 'Mumbai'
>
```

```
JS app.js > [?] city
1  const student = {
2    name: "karan",
3    age: 14,
4    class: 9,
5    subjects: ["hindi", "english", "math", "science"],
6    username: "karan@123",
7    password: "abcd",
8  };
9
10 let { username: user, password: secret, city = "Mumbai" } = student;
11
```

```

app.js / city
const student = {
  name: "karan",
  age: 14,
  class: 9,
  subjects: ["hindi", "english", "math", "science"],
  username: "karan@123",
  password: "abcd",
  city: "Pune",
};

let { username: user, password: secret, city = "Mumbai" } = student;

```

```

> city
< 'Pune'
>

```

```

app.js / ...
1  const student = {
2    name: "karan",
3    age: 14,
4    class: 9,
5    subjects: ["hindi", "english", "math", "science"],
6    username: "karan@123",
7    password: "abcd",
8  };
9
10 let { username: user, password: secret, city: place = "Mumbai" } = student;
11

```

```

top
> place
< 'Mumbai'
>

```